

CO1

ASSESSMENT TOOL-2

NAME : P. Ranjithkumar

DATE:13.07.2026

REG.NO : 192411119

MONDAY

COURSE CODE : CSA1103

COURSE NAME : OOAD

Question 1: Object-Oriented Design in Smart Healthcare Systems

Introduction

A Smart Healthcare System is used to manage patients, doctors, appointments, medical records, and billing. Object-Oriented Programming (OOP) helps in designing the system by representing real-world entities as classes and objects. OOP concepts make the system secure, reusable, scalable, and easy to maintain.

Application of Object-Oriented Concepts

1. Class

A class is a blueprint for creating objects.

Examples:

- Patient
- Doctor
- Appointment
- MedicalRecord

Each class contains attributes and methods related to that entity.

2. Object

An object is an instance of a class.

Example:

- Patient: Rahul
- Doctor: Dr. Kumar
- Appointment: 10:30 AM

Each object stores its own data while sharing the same class structure.

3. Encapsulation

Encapsulation combines data and methods into one unit and protects sensitive information.

Example:

Patient medical records are kept private and can only be accessed by authorized doctors.

Benefits:

- Protects patient data.
- Prevents unauthorized access.
- Improves security.

4. Inheritance

Inheritance allows one class to inherit properties from another class.

Example:

Person

└─ Patient

└─ Doctor

The Patient and Doctor classes inherit common details such as Name, Age, and Address from the Person class.

Benefits:

- Reduces duplicate code.
- Improves reusability.
- Simplifies maintenance.

5. Polymorphism

Polymorphism allows the same method to perform different tasks.

Example:

The generateReport() method produces different reports for a Cardiologist and a Neurologist.

Benefits:

- Improves flexibility.
- Supports code reuse.
- Makes software easy to extend.

Analysis

- Using Object-Oriented Design improves the healthcare system in many ways:
- Scalability: New modules such as Pharmacy or Laboratory can be added easily.
- Security: Encapsulation protects confidential patient information.
- Maintainability: Changes in one module do not affect other modules.
- Reusability: Existing classes can be reused in other healthcare applications.
- Reliability: Modular design reduces errors and improves software quality.

Conclusion

Object-Oriented concepts such as Class, Object, Encapsulation, Inheritance, and Polymorphism provide an efficient way to develop a Smart Healthcare System. These concepts improve security, scalability, maintainability, and code reusability, making the system reliable and easy to manage. This approach helps hospitals deliver better healthcare services while simplifying software development and future enhancements.

Question 2: Applying Encapsulation and Abstraction in Online Banking

Introduction

An Online Banking System allows customers to perform banking activities such as fund transfer, account management, balance enquiry, and transaction tracking. Since banking systems handle sensitive customer information, Object-Oriented Programming concepts like Encapsulation and Abstraction are essential for improving security and reducing system complexity.

Encapsulation

Encapsulation is the process of combining data and methods into a single unit while restricting direct access to sensitive information.

Example:

The BankAccount class stores customer details such as account number, balance, and PIN as private data. Customers can access or update this information only through authorized methods like deposit(), withdraw(), and checkBalance().

Benefits:

- Protects sensitive banking information.
- Prevents unauthorized access.
- Improves data security and integrity.

Abstraction

Abstraction hides the internal implementation and shows only the necessary features to the user.

Example:

When a customer transfers money, they only enter the account details and amount. The internal processes like verification, authentication, and transaction processing remain hidden.

Benefits:

- Reduces system complexity.
- Makes the application user-friendly.
- Simplifies software development.

Comparison

Encapsulation	Abstraction
Protects data	Hides implementation details
Improves security	Reduces complexity
Uses private data members	Uses abstract classes/interfaces

Analysis

- Encapsulation secures customer information.
- Abstraction simplifies banking operations.
- Both concepts improve software reliability.
- They reduce maintenance effort.
- Banking systems become secure and easy to use.

Conclusion

Encapsulation and Abstraction are important OOP concepts in Online Banking Systems. Encapsulation protects customer data from unauthorized access, while Abstraction simplifies banking operations by hiding internal complexity. Together they improve security, usability, and maintainability.

Question 3: Modeling Object Relationships in a University Management System

Introduction

A University Management System manages students, faculty, departments, and courses. Object relationships help represent how different objects interact with each other. The main relationships are Association, Aggregation, and Composition.

Association

Association represents a relationship where two objects communicate but can exist independently.

Example:

A Student enrolls in a Course.

Student ----- Course

Aggregation

Aggregation represents a "has-a" relationship where one object contains another, but both can exist independently.

Example:

Department

|

Faculty

A Faculty belongs to a Department, but the Faculty can exist even if the Department is removed.

Composition

Composition represents a strong relationship where the child object cannot exist without the parent.

Example:

Student

|

Student ID Card

If the Student is deleted, the ID Card is also deleted

Analysis

- Association improves communication between objects.
- Aggregation promotes code reuse.
- Composition provides strong ownership.

- Proper relationships reduce complexity.
- Software becomes flexible and maintainable.

Conclusion

Association, Aggregation, and Composition help model real-world relationships in a University Management System. These relationships improve software organization, flexibility, maintainability, and reusability, making the system easier to develop and manage.

Question 4: Importance of SDLC in E-Commerce Development

Introduction

An E-Commerce Platform manages products, customers, payments, and order tracking. The Software Development Life Cycle (SDLC) provides a structured process for developing reliable and high-quality software.

Phases of SDLC

1. Requirement Analysis

Requirements are collected from customers and stakeholders.

2. System Design

The system architecture, database, and user interface are designed.

3. Coding

Developers convert the design into source code.

4. Testing

The software is tested to identify and fix defects.

5. Deployment

The completed software is released to customers.

6. Maintenance

The software is updated, improved, and maintained after deployment.

Analysis

- Requirement Analysis ensures customer needs are understood.
- Design creates a strong software architecture.
- Coding develops the application.
- Testing improves software quality.
- Deployment delivers the application to users.
- Maintenance keeps the software updated and reliable.

Advantages

- Improves software quality.
- Reduces project risks.
- Increases customer satisfaction.
- Makes software easy to maintain.
- Ensures systematic development.

Conclusion

The SDLC provides a structured approach for developing E-Commerce applications. Each phase contributes to software quality, reliability, and customer satisfaction, ensuring that the final product is secure, efficient, and easy to maintain.

Question 5: Selecting an Appropriate SDLC Model for Mobile App Development

Introduction

A Mobile Learning Application is developed for students to access online courses, quizzes, assignments, and video lectures. Since customer requirements change frequently, selecting the appropriate Software Development Life Cycle (SDLC) model is very important. Three commonly used SDLC models are Waterfall, Agile, and Spiral. Each model has its own advantages and limitations depending on the project requirements.

Waterfall Model

The Waterfall Model is a linear and sequential software development model. Each phase is completed before moving to the next phase.

Advantages

- Simple and easy to understand.
- Suitable for projects with fixed requirements.
- Easy to manage and document.
- Well-defined development phases.

Limitations

- Difficult to accommodate changing requirements.
- Customer feedback is received only after development.
- Errors are identified late.
- Not suitable for dynamic projects.

Agile Model

The Agile Model is an iterative software development approach where software is developed in small increments with continuous customer feedback.

Advantages

- Easily handles changing requirements.
- Continuous customer involvement.
- Faster delivery of software.
- Early testing reduces defects.
- Improves software quality.

Limitations

- Requires experienced development teams.
- Continuous customer participation is necessary.
- Documentation may be less detailed.

Spiral Model

The Spiral Model combines iterative development with risk analysis. It focuses on identifying and reducing project risks at every stage.

Advantages

- Excellent risk management.
- Suitable for large and complex projects.
- Customer feedback at every iteration.
- Flexible development process.

Limitations

- Expensive to implement.
- Requires expert risk analysis.
- Not suitable for small projects.

Analysis

Model	Best Suitable For	Limitation
Waterfall	Fixed requirements	Cannot handle frequent changes
Agile	Frequently changing requirements	Requires continuous customer involvement
Spiral	Large and high-risk projects	High development cost

Conclusion

Among the three SDLC models, Agile is the best choice for developing a Mobile Learning Application because it allows frequent updates, rapid development, and continuous customer interaction. It improves software quality, customer satisfaction, and flexibility while reducing development risks.

Question 6: Modular Design in a Hospital Management System

Introduction

A Hospital Management System manages patients, doctors, appointments, billing, pharmacy, and laboratory services. If all these functions are tightly connected, maintaining and updating the software becomes difficult. Modular Design using Object-Oriented Principles solves these problems by dividing the system into independent modules.

Existing Problems

- Tightly coupled modules.
- Difficult to update software.
- High maintenance cost.
- Code duplication.
- Poor scalability.
- Difficult debugging.

Object-Oriented Solution

The system can be divided into separate classes such as:

- Patient
- Doctor
- Appointment
- Pharmacy
- Billing

Object-Oriented Principles Used

Encapsulation

Protects patient records and billing information from unauthorized access.

Inheritance

Doctor and Patient classes inherit common details from the Person class.

Polymorphism

Different billing methods can calculate charges differently for OP, IP, and Emergency patients.

Analysis

- Independent modules simplify maintenance.
- New hospital services can be added easily.
- Code becomes reusable.
- Software performance improves.
- Debugging becomes easier.
- Development time is reduced.

Advantages

- Better maintainability.
- Improved scalability.
- Increased code reusability.
- Easier software testing.
- Reduced development cost.
- Improved software quality.

Conclusion

Using modular design and object-oriented principles makes the Hospital Management System more flexible, maintainable, and scalable. Independent modules reduce complexity and improve overall software quality.

Question 7: Enhancing Software Quality Using Design Patterns

Introduction

A Hotel Reservation System manages room booking, customer details, payments, and notifications. Due to duplicated code and inefficient object creation, software performance decreases. Design Patterns provide reusable solutions to these common software design problems.

Factory Pattern

The Factory Pattern creates objects without exposing the object creation logic.

Example:

Different room types such as Standard Room, Deluxe Room, and Suite Room can be created using a Factory class.

Benefit

- Reduces duplicate code.
- Simplifies object creation.

Singleton Pattern

Singleton ensures that only one object exists throughout the application.

Example:

Database Connection or Hotel Configuration Manager.

Benefit

- Saves memory.
- Ensures consistent database access.

Observer Pattern

Observer Pattern allows multiple objects to receive notifications automatically.

Example:

Customers receive SMS or Email notifications when their booking status changes.

Benefit

- Automatic communication.
- Loose coupling between modules.

Analysis

- Eliminates duplicate code.
- Improves software flexibility.
- Makes maintenance easier.

- Simplifies object creation.
- Enhances software quality.
- Improves code reusability.

Conclusion

Factory, Singleton, and Observer Patterns improve the Hotel Reservation System by reducing code duplication, simplifying object management, and enhancing communication between modules. These design patterns make the software reliable, maintainable, and scalable.

Question 8: UML-Based Modeling for an Online Examination System

Introduction

An Online Examination System manages students, instructors, examinations, question papers, and result processing. UML (Unified Modeling Language) helps developers visualize, design, and document the software system before implementation.

1. Use Case Diagram

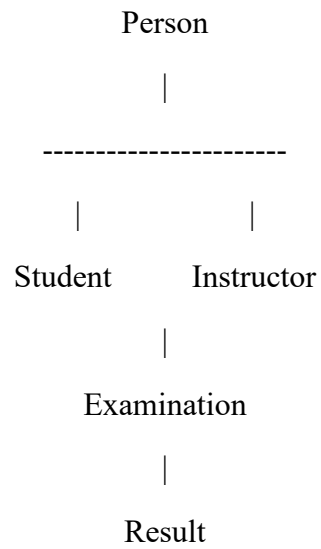
Actors

- Student
- Instructor
- Administrator

Use Cases

- Login
- Attend Examination
- Upload Questions
- Evaluate Answers
- View Results
- Manage Users

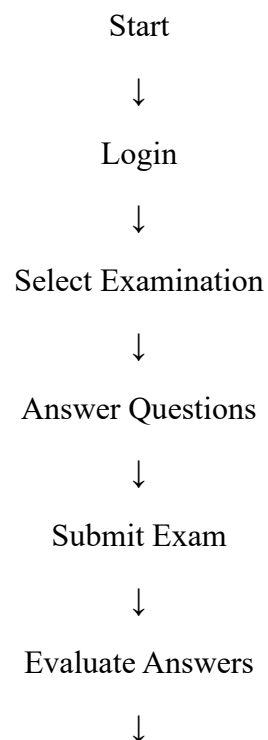
2. Class Diagram



Classes include:

- Student
- Instructor
- Examination
- Question
- Result

3. Activity Diagram



Display Result



End

Analysis

- UML improves system visualization.
- Simplifies requirement analysis.
- Enhances communication between developers and clients.
- Reduces design errors.
- Improves software documentation.
- Makes future maintenance easier.

Advantages

- Better software planning.
- Clear system documentation.
- Easy stakeholder communication.
- Improved software quality.
- Supports object-oriented development.

Conclusion

UML diagrams such as Use Case, Class, and Activity Diagrams provide a clear representation of the Online Examination System. They help developers understand system requirements, improve communication, reduce design errors, and develop reliable, maintainable, and scalable software systems.